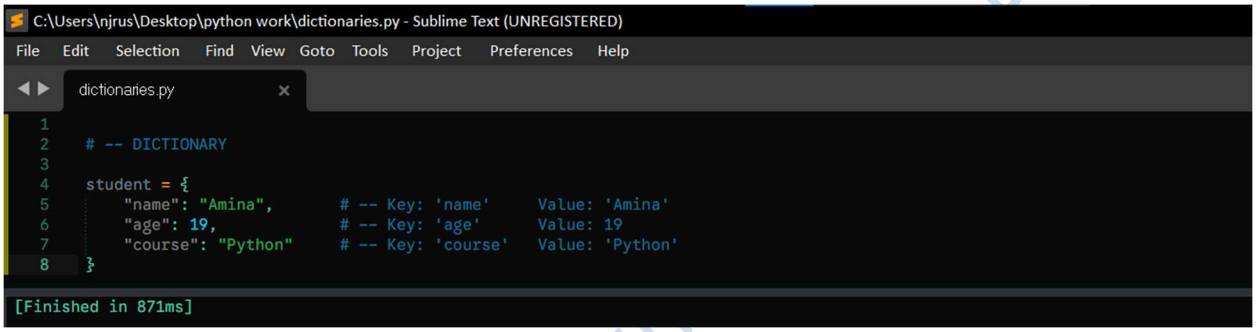
Dictionaries in Python

In Python, we often need to store related information so a program can use it later. Lists are useful when we want to keep a collection of items in order. But what if each piece of information has a label that helps us identify it? For example, **a student's name**, **age**, and **course** are different pieces of information with different meanings. A dictionary lets us organize such information using labels and their associated values.

A **dictionary** is a built-in Python data structure that stores information as **key–value** pairs. Each key is used to **identify** or **look up** its associated value. You can think of an ordinary dictionary: you look up a word to find its meaning. In a Python dictionary, you use a key to find the value connected to it.



This dictionary contains three key–value pairs:

Key	Value	Meaning
"name"	"Amina"	The student's name
"age"	19	The student's age
"course"	"Python"	The course the student is taking

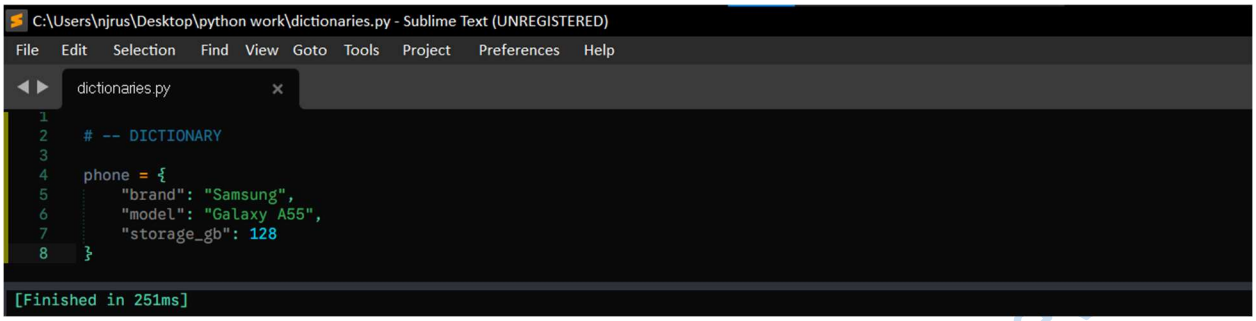
Note: The key tells us what the value represents. It is not the real value you want to work with in your programs.

Dictionary Definition

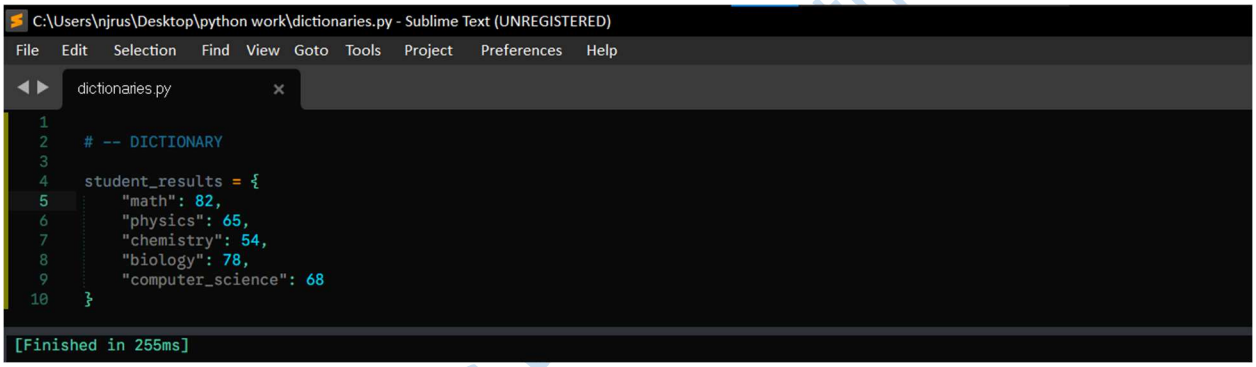
A dictionary is usually created with curly braces, {}. Each key is followed by a colon and its value, and commas separate one key–value pair from the next.

```
dictionary_name = {
    key1: value1,
    key2: value2,
    key3: value3
}
```

Note that this is a general pattern. In actual code, do replace the example keys and values with real data. See below:

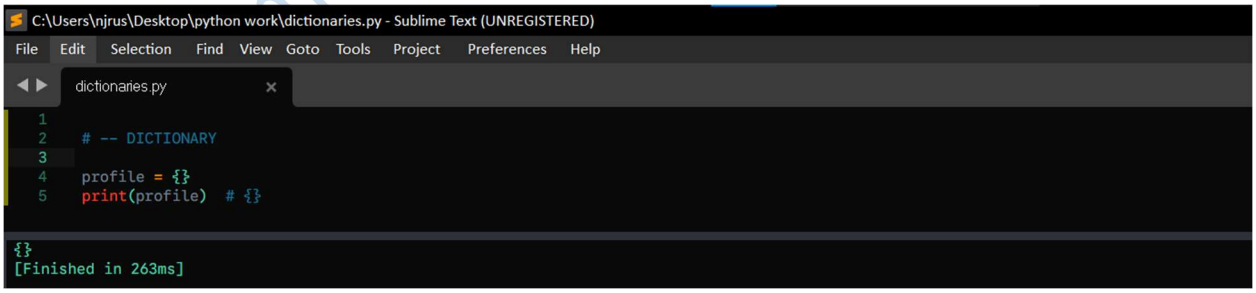


This dictionary stores details about a phone. The keys "brand", "model", and "storage_gb" identify the type of information stored in each value.



This dictionary stores results for a certain student in five units. The keys "math", "physics", "chemistry", "biology" and "computer_science" help in getting the score on that unit.

You can also create a dictionary before it contains any information. This is called an empty dictionary. In Python, {} creates an empty dictionary.

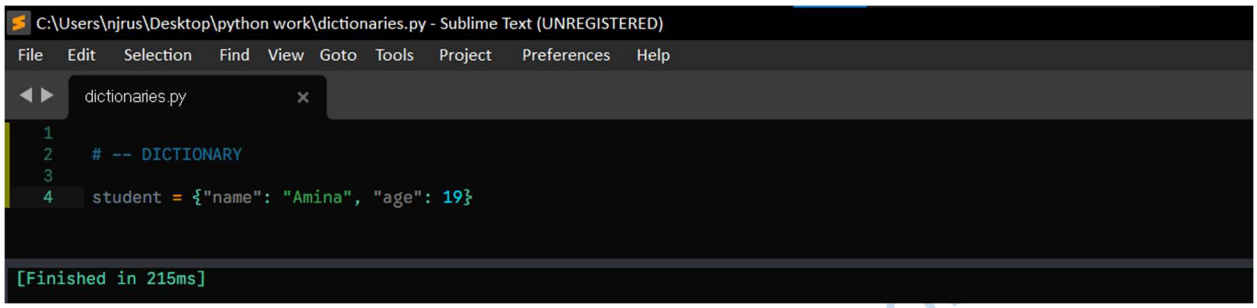


An empty dictionary can be useful when a program will add information later.

Some Key Characteristics of Dictionaries

1. They store key–value pairs

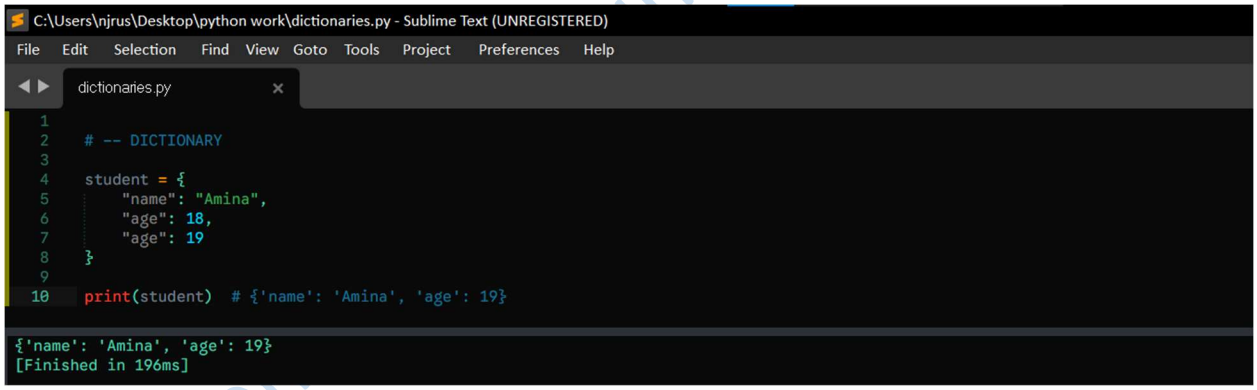
Recall that a list lets us identify an item by its position. On the other hand, dictionaries let us associate a value with a meaningful key, which can make structured information easier to understand.



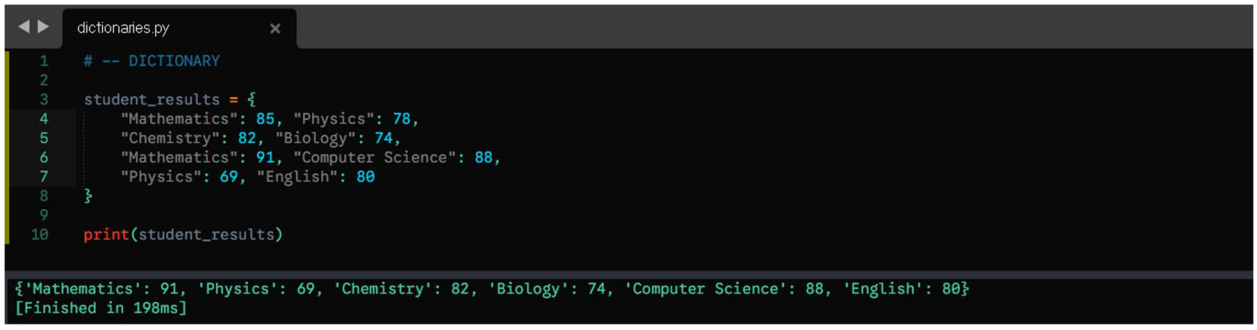
Given this, you can understand that whoever this profile belongs to, their **name** is **Amina** and their **age** is **19**.

2. Keys are unique

A dictionary **cannot keep two separate entries with the same key**. If the same key appears more than once when a dictionary is created, **the later value replaces** the earlier value.



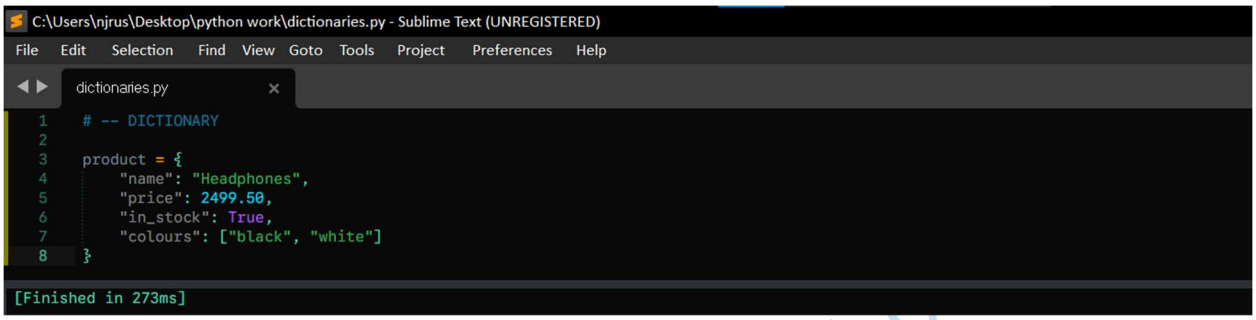
The resulting dictionary has one "age" key, with value 19. You'll need to choose distinct keys for different pieces of information to avoid losing data in your programs.



Observe how duplicate dictionary keys can overwrite earlier values, potentially causing original marks to be lost. This can lead to incorrect semester averages, final grades, and academic decisions.

3. Values can have different data types

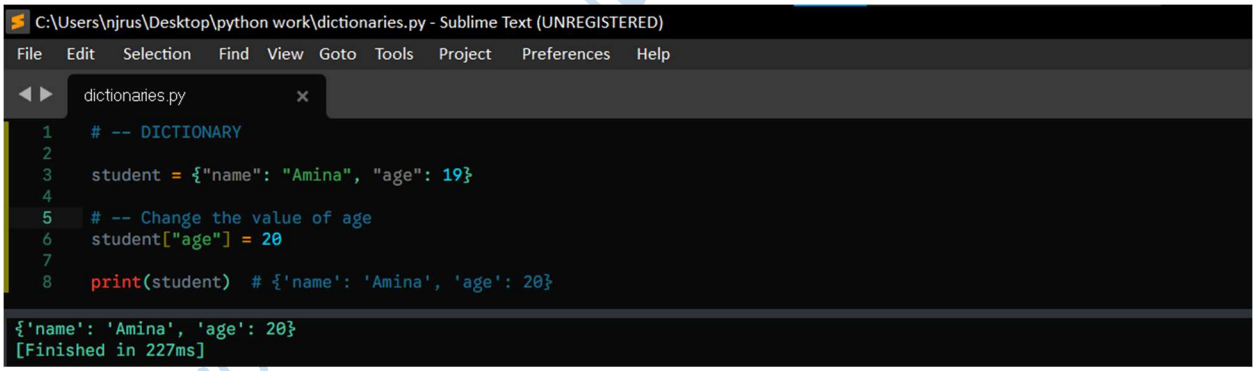
A dictionary's values do not all need to be the same type. Values can be strings, integers, floats, Boolean values, lists, and even other dictionaries.



This dictionary contains a string, a floating-point number, a Boolean, and a list. That flexibility helps dictionaries represent real-world objects and their details.

4. Dictionaries are mutable

Mutable means an object can be changed after it has been created. A dictionary can have new pairs added, existing values updated, or entries removed. These operations will be covered in later lessons.



This example changes the value associated with the "age" key.

Note: Changing the values associated with a certain key makes updates to the existing dictionary. A new dictionary is not created.

5. Dictionaries preserve insertion order

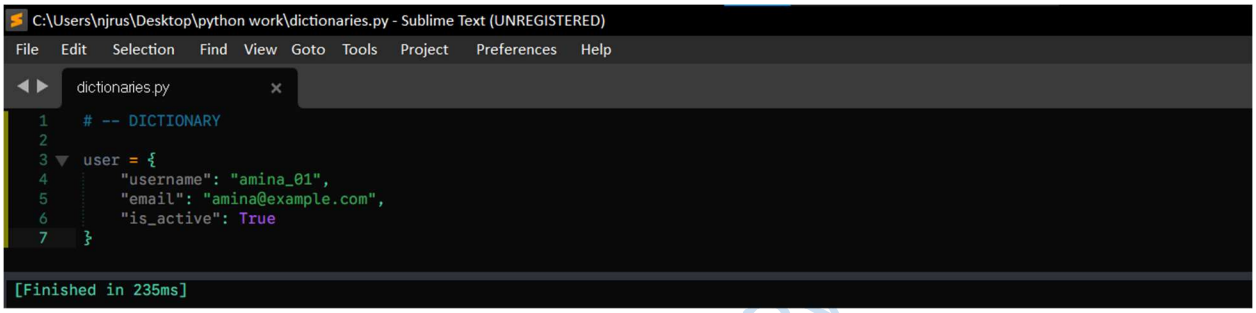
Modern Python dictionaries preserve the order in which key–value pairs are inserted. However, dictionaries are mainly used to look up values by key, rather than to access items by numeric position as with a list.

Note: Strings and numbers are common dictionary keys, but mutable types such as lists cannot be used as keys. Values are more flexible and can include lists or dictionaries or any other data structure. At this level, use strings as your keys as python can use them reliably to identify an entry.

Everyday Programming Uses

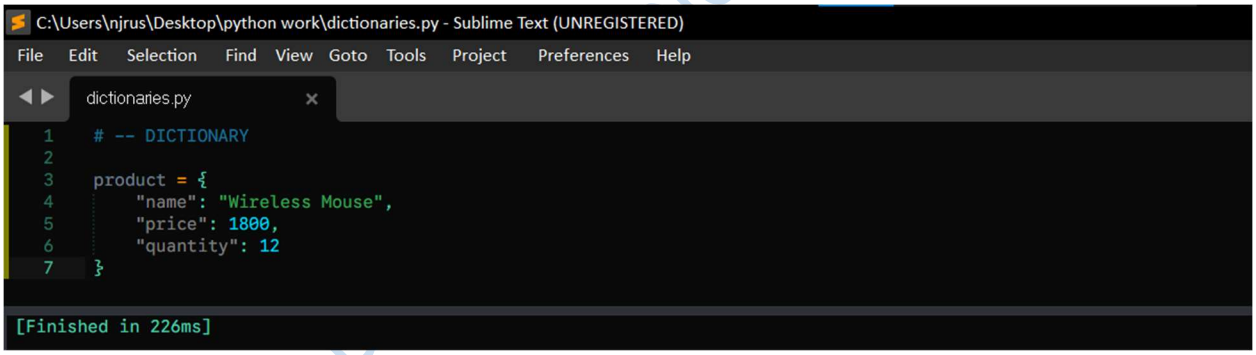
User profiles

A website or application can represent a user's username, email address, and account status.



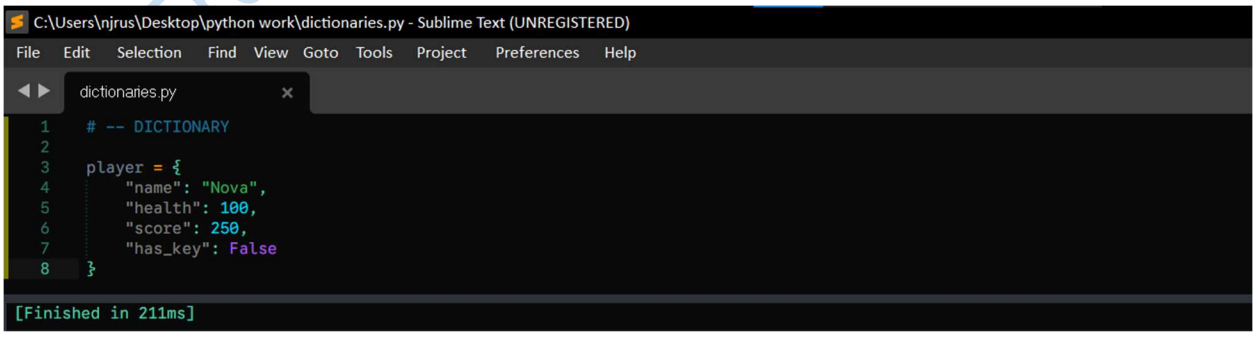
Product details in an online shop

A shop can store a product's name, price, and available quantity together.



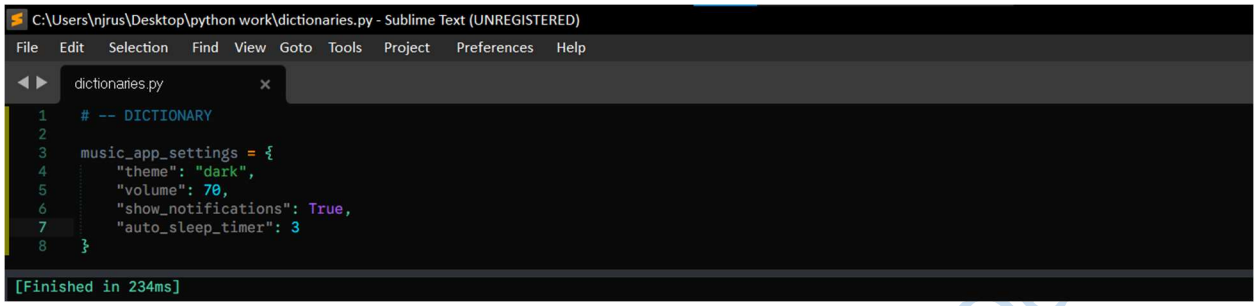
Game settings or player statistics

A game can keep track of a player's health, score, and other game state.



Configuration settings

An application can store preferences such as theme, volume level, and whether notifications are enabled or the timer to stop the music automatically.



We will be looking at dictionaries in detail now in the upcoming lessons.

From Basics to Junior Python